Section 10.3: Resiliency & Chaos Engineering

> ***Study Guide*** *| 3 Files in This Series | Section 10.**

Executive Overview

Resiliency engineering assumes failure is inevitable and designs systems that degrade gracefully rather than collapse completely. Chaos engineering tests those assumptions proactively, before production incidents do. This section covers the core fault-tolerance patterns, the scientific method of chaos experiments, and disaster recovery strategy — all with explicit guidance on when and where to apply each approach.

---

1. Fault Tolerance Patterns

Circuit Breaker

The circuit breaker prevents a failing dependency from cascading into a full system failure. It is a proxy that monitors calls to a dependency and "opens" when failure exceeds a threshold, returning fast failures instead of waiting for timeouts.

State machine:

```
CLOSED   →  (failures ≥ threshold)     →  OPEN
OPEN     →  (timeout elapsed)          →  HALF-OPEN
HALF-OPEN → (probe succeeds)           →  CLOSED
HALF-OPEN → (probe fails)              →  OPEN
```

Full Java implementation:

```
class CircuitBreaker {
    private State state = State.CLOSED;
    private int failureCount = 0;
    private long lastFailureTime;
    private final int failureThreshold;
    private final long timeout; // ms before retrying
```

```
    public Result call(Supplier<Result> operation) {
        if (state == State.OPEN) {
            if (System.currentTimeMillis() - lastFailureTime > timeout) {
                state = State.HALF_OPEN;  // Allow one probe
            } else {
                throw new CircuitBreakerOpenException("Fast fail — dependency
unhealthy");
            }
        }

        try {
            Result result = operation.get();
            onSuccess();
            return result;
        } catch (Exception e) {
            onFailure();
            throw e;
        }
    }

    private void onSuccess() {
        failureCount = 0;
        state = State.CLOSED;
    }

    private void onFailure() {
        failureCount++;
        lastFailureTime = System.currentTimeMillis();
        if (failureCount >= failureThreshold) {
            state = State.OPEN;
        }
    }
}
```

When to use: Any synchronous call to a remote dependency (database, third-party API, downstream service). Do not use for idempotent reads where retries are harmless and cheap.

Bulkhead Pattern

Bulkheads isolate failure domains by giving each category of work its own resource pool. If one pool is exhausted, other pools continue normally.

```
class BulkheadExecutor:
    def __init__(self):
        # Critical path: large pool → low wait time
        self.critical_pool  = ThreadPoolExecutor(max_workers=10)

        # Bulk background work: small pool → bounded resource use
        self.bulk_pool      = ThreadPoolExecutor(max_workers=2)
```

```
        # Reporting: minimal pool → won't starve other work
        self.reporting_pool = ThreadPoolExecutor(max_workers=1)

    def execute_critical(self, func):
        return self.critical_pool.submit(func)

    def execute_bulk(self, func):
        return self.bulk_pool.submit(func)
```

Why it matters: Without bulkheads, a surge of slow bulk operations fills the shared thread pool, blocking critical operations. The 2-thread bulk pool can be 100% busy without affecting critical throughput.

When to use: Any system where different workload classes share infrastructure — API endpoints (critical vs analytics), database queries (transactional vs reporting), message processing (priority queues).

Retry with Exponential Backoff + Jitter

Retrying immediately after a failure often worsens the problem (thundering herd). Exponential backoff spreads retries out; jitter prevents synchronised retry storms.

```
import random, time

def exponential_backoff_retry(func, max_attempts=5):
    for attempt in range(max_attempts):
        try:
            return func()
        except TransientError:
            if attempt == max_attempts - 1:
                raise  # Final attempt — propagate

            # Base delay doubles each attempt, capped at 30s
            base_delay_ms = min(1000 * (2 ** attempt), 30_000)

            # Jitter: ±10% of base delay to desynchronise retries
            jitter_ms = random.uniform(-0.1 * base_delay_ms, 0.1 * base_delay_ms)

            time.sleep((base_delay_ms + jitter_ms) / 1000)

# Delay schedule (approximate):
# Attempt 0 → immediate
# Attempt 1 → ~1s
# Attempt 2 → ~2s
# Attempt 3 → ~4s
# Attempt 4 → ~8s  (final)
```

When to use: Idempotent operations against transient failures (network blips, rate limiting, momentary overload). Do **not** retry non-idempotent operations (payment charge, order creation) without deduplication logic.

Pattern Comparison — When to Use What

Pattern	Problem it solves	Do not use when
Circuit breaker	Cascading failure from slow/failing dependency	Dependency is local (in-process); failure is permanent
Bulkhead	Resource starvation from one workload class	All workloads have equal criticality
Retry + backoff	Transient failures (network, rate limits)	Operation is non-idempotent without dedup
Timeout	Unbounded waiting for slow dependencies	Operation has no acceptable upper bound
Fallback/default	Graceful degradation when dependency unavailable	Degraded behaviour is unacceptable to users

Layering: These patterns compose. A well-designed client combines **timeout → retry with backoff → circuit breaker → fallback** in that order.

---

2. Chaos Engineering

The Scientific Method Applied to Production

Chaos engineering is not “break things randomly.” It is a controlled experiment with a hypothesis, measurement, and conclusion.

Experiment template:

```
1. Define steady state
   Metric: 99th-percentile latency < 200ms, error rate < 0.1%

2. Hypothesize behaviour under failure
   "If we inject 10% packet loss to the payment service,
    the circuit breaker will absorb the failures and
    overall error rate will remain below 0.5%"
```

```
3. Inject controlled failure
   Tool: Gremlin network latency attack — 200ms + 10% loss
   Duration: 5 minutes
   Blast radius: 5% of traffic (canary)

4. Observe
   - Did error rate stay below 0.5%?
   - Did circuit breaker open as expected?
   - Did latency p99 stay below threshold?

5. Conclude and act
   Hypothesis confirmed → schedule regular run, expand blast radius
   Hypothesis violated → find gap, fix, retest
```

Chaos Tools Comparison

Tool	Failure types	Where it runs	Best for
Chaos Monkey	Instance termination	AWS Auto Scaling Groups	Testing auto-recovery of stateless services
Gremlin	Network, CPU, memory, state	Any host/container	Precise controlled experiments, compliance
AWS Fault Injection Simulator	AWS-native failures (AZ, API)	AWS only	Cloud-native resilience testing
Chaos Mesh	Pod kill, network, IO	Kubernetes	k8s-native experiments, GitOps integration
Litmus	Kubernetes + cloud	Kubernetes	Open-source, good for CI integration

When to run chaos experiments: - After implementing a new resilience pattern (verify it works) - Before a major traffic event (Black Friday, product launch) - As part of quarterly game days - In CI/CD pipelines for critical services (automated, scoped to staging)

When NOT to run: - On production without a rollback plan - Without stakeholder awareness - Before establishing a steady-state baseline

---

3. Disaster Recovery Strategies

RTO and RPO — Worked Definitions

Recovery Time Objective (RTO): The maximum time the system can be unavailable before the outage causes unacceptable business damage.

Recovery Point Objective (RPO): The maximum amount of data loss that is acceptable, measured as a time window.

Example targets by service type:

```
Payment processing:
  RTO = 15 minutes  (revenue loss per minute is high)
  RPO = 0           (no transaction can be lost)

Customer portal:
  RTO = 4 hours     (users can retry later)
  RPO = 1 hour      (lose a session, not an account)

Analytics pipeline:
  RTO = 24 hours    (internal tool, low urgency)
  RPO = 24 hours    (yesterday's report is acceptable)
```

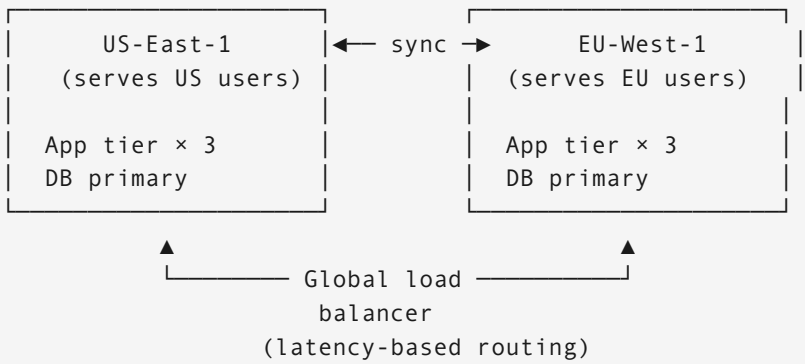
DR Strategy Spectrum

Strategy	RTO	RPO	Relative cost	Complexity	Use case
Backup & Restore	Hours	Last backup	$	Low	Archives, dev environments
Pilot Light	10–30 min	Seconds	|*Low – Medium*| *Internaltools*, *moderateSLOs*||* * *WarmStandby* * *|2–10*min*|*Seconds*|$	Medium	Customer-facing, 99.9% SLO
Hot Standby (Active-Active)	Seconds	None	$$$$	High	Payments, auth, 99.99% SLO

Pilot Light explained: Core data replication runs continuously, but compute is off. On failover, compute starts (5–10 min) and points at the replicated data. Cheap because idle compute costs nothing.

Warm Standby explained: A scaled-down but live replica runs in the DR region, receiving all writes. Failover is just a DNS swap + scale-up, not a cold start.

Multi-Region Active-Active Architecture



Conflict resolution: When both regions accept writes to the same record (e.g., user updates profile in both regions simultaneously): - **Last-write-wins** (simple, loses data) - **Version vectors** (complex, preserves both) - **Business rules** (e.g., "higher loyalty tier wins")

When to use active-active: Only when both availability (RTO ≈ 0) AND global latency requirements are present. The operational complexity is high — conflicts, split-brain scenarios, and replication lag all require explicit handling.

---

4. Interview Problems

Problem 1 — Multi-Provider Circuit Breaker for Payment Processing

Scenario: Your checkout service calls three payment providers (Stripe, PayPal, Braintree). Any one can be down at any time. Design a resilient payment processor that handles partial failures without dropping orders.

Solution:

Step 1 — Independent circuit breaker per provider:

```
class PaymentService:
    def __init__(self):
        self.breakers = {
            "stripe":    CircuitBreaker(failure_threshold=5, timeout_sec=60),
            "paypal":    CircuitBreaker(failure_threshold=5, timeout_sec=60),
```

```
            "braintree": CircuitBreaker(failure_threshold=5, timeout_sec=60),
        }
```

Why independent breakers? A shared breaker would open when Stripe fails, blocking PayPal even though PayPal is healthy. Each provider's health is independent.

Step 2 — Ordered fallback chain:

```
def process_payment(amount, card):
    provider_order = ["stripe", "paypal", "braintree"]
    errors = {}

    for provider in provider_order:
        breaker = self.breakers[provider]

        if breaker.is_open():
            continue  # Skip without incrementing failure count

        try:
            return breaker.call(
                lambda p=provider: self._charge(p, amount, card)
            )
        except Exception as e:
            errors[provider] = str(e)
            # Breaker internally counts this failure

    # All providers failed — queue for retry rather than losing the order
    self._queue_for_retry(amount, card, errors)
    raise AllProvidersFailed(errors)
```

Step 3 — Bulkhead to protect the database:

```
# Payment processing must not exhaust DB connection pool
payment_semaphore = asyncio.Semaphore(50)

async def process_payment_safe(amount, card):
    async with payment_semaphore:  # Max 50 concurrent payments
        return await process_payment(amount, card)
```

Step 4 — Distributed circuit state (prevents thundering herd):

```
# Share breaker state across all instances via Redis
# When one instance trips a breaker, all instances see it

def sync_breaker_state(provider, state):
    redis.setex(
        f"circuit:{provider}",
        json.dumps({"state": state, "updated": time.time()}),
        ttl=120  # 2-minute TTL — stale state resolves itself
```

```
    )

# Half-open: only 10% of requests probe the recovering provider
def allow_probe(provider):
    if self.breakers[provider].state == State.HALF_OPEN:
        return random.random() < 0.10  # 10% probe rate
    return True
```

Step 5 — Degraded mode with durable queuing:

```
def _queue_for_retry(amount, card, errors):
    # Payment is not lost — stored durably for retry
    queue.enqueue({
        "amount": amount,
        "card_token": card.token,  # Never store raw card data
        "errors": errors,
        "enqueued_at": time.time()
    }, ttl=3600)  # Retry window: 1 hour

    # Return 503 with Retry-After header
    raise ServiceUnavailable(
        "Payment temporarily unavailable. Order saved — retrying automatically.",
        retry_after=30
    )
```

Alerting:

```
Circuit opened on any provider    → Warning (single provider failure)
All providers open simultaneously → Page (total payment failure)
Retry queue depth > 100           → Page (orders at risk)
```

Key insight: The fallback chain means one provider failure is invisible to users. Two providers failing degrades to queued retry (user sees "try again later"). Only total failure of all three triggers an error — and even then the order is not lost.

Problem 2 — Designing a Chaos Experiment for Auto-Scaling

Scenario: You have implemented auto-scaling for your API tier. You claim it handles 3× traffic spikes within 2 minutes. Design a chaos experiment to validate this claim.

Solution:

Step 1 — Define steady state precisely:

```
Baseline (last 7 days):
  p99 latency:      85ms
```

```
  Error rate:       0.02%
  Instances:        4
  CPU utilisation:  35%

Acceptance threshold for experiment:
  p99 latency:  < 200ms  (2.4× headroom)
  Error rate:   < 0.5%   (25× headroom)
  Scale-out:    complete within 120 seconds
```

Step 2 — Hypothesis:

```
"When traffic increases 3× from baseline for 5 minutes,
 auto-scaling will add 8 instances within 2 minutes,
 p99 latency will remain below 200ms,
 and error rate will remain below 0.5%."
```

Step 3 — Experiment design:

```
Tool: Artillery (load generator) + AWS Fault Injection Simulator

Phase 1: Baseline (5 min) — normal traffic, capture metrics
Phase 2: Spike (5 min)    — 3× traffic injected via load generator
Phase 3: Recovery (5 min) — return to baseline, confirm scale-in

Blast radius:
  Start in staging (100% of experiment traffic)
  If successful: run in production with 5% of real traffic (canary)

Monitoring during experiment:
  - CloudWatch: instance count, CPU, ALB latency
  - Prometheus: p99 latency, error rate
  - Auto-scaling activity log: time-to-first-instance, time-to-full-scale
```

Step 4 — Failure analysis matrix:

```
Scenario A: Scale-out takes 4 minutes (not 2)
  → Auto-scaling cooldown too conservative; reduce scale-in cooldown
  → Pre-warm instances before expected spikes

Scenario B: p99 spikes to 450ms despite correct instance count
  → Bottleneck is not compute — check DB connection pool, cache miss rate

Scenario C: Error rate hits 2% during first 90 seconds of spike
  → Application startup time too slow; add readiness probe tuning
     or switch to pre-warmed instance pool
```

Problem 3 — Choosing a DR Strategy for a Healthcare SaaS

Scenario: You are designing DR for a healthcare SaaS with three tiers: patient-facing portal, clinical data API, and analytics dashboard. Each has different business requirements. Propose a DR strategy for each tier.

Solution:

Analysis by tier:

```
Patient portal (appointment booking):
  Business impact of 4hr outage: appointments missed, poor NPS
  RTO target: 30 minutes
  RPO target: 5 minutes (last appointment booking)
  Data volume: low (session + booking records)
  → Warm Standby in second region

Clinical data API (accessed by medical staff):
  Business impact of outage: clinical workflow blocked
  Regulatory: HIPAA — data must not be lost
  RTO target: 5 minutes
  RPO target: 0 (no record can be lost)
  → Active-Active multi-region with synchronous replication

Analytics dashboard (accessed by hospital admin):
  Business impact of 24hr outage: delayed reporting only
  RTO target: 24 hours
  RPO target: 24 hours
  → Backup & Restore with daily snapshots to S3
```

Cost justification:

```
Active-Active (clinical API): 4× cost, justified by regulatory risk
Warm Standby (portal):        2× cost, justified by patient experience SLA
Backup & Restore (analytics): 1× cost, acceptable for internal tooling
```

Key insight: Not every component needs the same DR tier. Over-engineering analytics to active-active wastes money; under-engineering the clinical API creates regulatory and patient safety risk. Map each tier to business impact, then select the cheapest strategy that meets that impact threshold.

Quick-Reference Cheat Sheet

```
Circuit breaker states:  CLOSED → OPEN → HALF-OPEN → CLOSED
Bulkhead:                Separate thread pools by workload class
Retry:                   Exponential backoff + jitter; only for idempotent ops
```

```
Chaos method:           Hypothesis → inject → observe → conclude
RTO:                    Max acceptable downtime
RPO:                    Max acceptable data loss
DR tiers (cost order):  Backup < Pilot Light < Warm Standby < Active-Active
Active-Active:          Only when RTO ≈ 0 AND global latency required
```